



Sistemas Embebidos I

Otoño 2026

**Práctica #6: Serial Communication & Scheduling:
UART**

María Fernanda Conde Calatayud

Andrea Ceja Lara

Kiria Margalli Chumacero

Jorge Emiliano Sánchez Rodríguez

09/09/26

Índice

<i>Práctica #6: Serial Communication & Scheduling: UART.....</i>	2
Introducción	2
Descripción de la práctica	2
Objetivo	2
Marco teórico	2
Desarrollo	3
Materiales	3
Esquemático	3
Desarrollo.....	4
Resultados	5
Evidencias	6
Conclusiones	7

Práctica #6: Serial Communication & Scheduling: UART

Introducción

Descripción de la práctica

La práctica consiste en desarrollar un programa en un sistema embebido que gestione de manera simultánea la comunicación serial por hardware (**UART**) y la ejecución de tareas periódicas mediante un **scheduler cooperativo**.

Objetivo

Implementar la interfaz de comunicación UART mediante interrupciones y buffers circulares dentro de un esquema de tareas de tiempo compartido (scheduler cooperativo), analizando y midiendo experimentalmente los límites temporales del sistema para evitar la pérdida de información.

Marco teórico

UART (Universal Asynchronous Receiver-Transmitter): Protocolo de comunicación serial asíncrono que transmite datos bit a bit a través de un solo cable (TX a RX) con una referencia común (GND). Dado que carece de línea de reloj compartida, ambos dispositivos deben acordar previamente parámetros como la velocidad (baudrate) y el formato del paquete.

Formato de trama 8N1: Protocolo estándar donde un byte (8 bits de datos) se encapsula entre 1 bit de inicio (*start*) y 1 bit de parada (*stop*), sin bit de paridad. Cada byte requiere la transmisión de 10 bits en total, lo que genera un sobrecosto del 20% en el canal de comunicación.

FIFO e Interrupciones (ISR): El periférico UART en el microcontrolador cuenta con un buzón FIFO por hardware de 32 bytes. A 115200 baudios, un byte tarda aproximadamente $86.8 \mu s$ en llegar, lo que otorga una ventana temporal máxima de unos 2.8 ms ($32 \times 86.8 \mu s$) para leer el hardware antes de provocar un *overflow*. Para responder de manera inmediata, se utiliza una interrupción (ISR) que extrae los bytes del hardware y los almacena en memoria.

Buffer Circular: Estructura de datos basada en un arreglo de tamaño fijo (ej. 256 bytes) que utiliza dos punteros independientes: head (movido únicamente por la ISR al escribir) y tail (movido por el bucle principal al leer). Esto permite desacoplar la velocidad de recepción respecto a la velocidad de procesamiento del programa.

Scheduler Cooperativo: Técnica de multitarea donde las tareas se ejecutan secuencialmente en el bucle principal. Cada tarea se implementa como una función que

verifica si ha transcurrido su tiempo de ejecución (mediante comparaciones con marcas de tiempo como `time_us_32()`) y cede el control voluntariamente sin bloquear el microcontrolador con funciones tipo `sleep_ms()`.

Tipos de Errores de Comunicación:

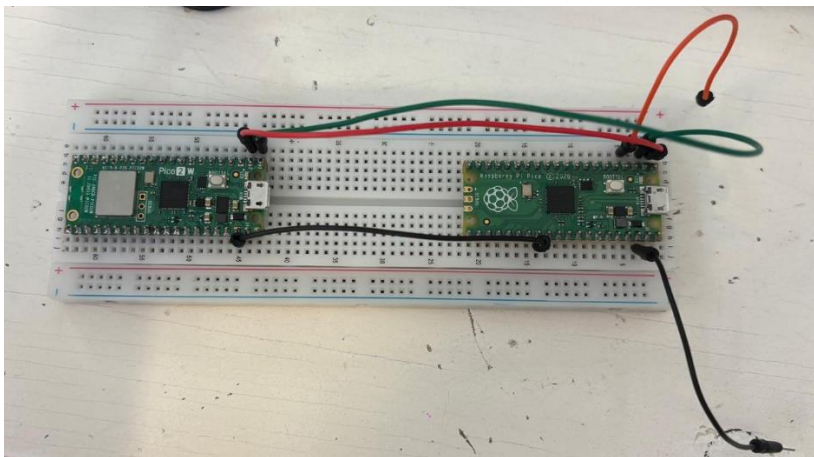
- **Overrun Error:** Ocurre cuando el software es lento y la FIFO de hardware o el buffer de la aplicación se llena por completo sin haber procesado los bytes previos.
- **Framing Error:** Ocurre por problemas de capa física o desacople de reloj/velocidad (mayor al $\pm 3\%$), haciendo que el receptor no encuentre el bit de parada (*stop*) donde lo esperaba.

Desarrollo

Materiales

- Raspberry Pi Pico
- Raspberry Pi Pico W
- Osciloscopio
- Jumpers

Esquemático



Desarrollo

Código raspberry pi pico (principal)

```

1  // ===== UART =====
2  static volatile uint8_t rx_buf[...], tx_buf[...]; // buffers circulares
3
4  void on_uart_rx(void) {
5  |    // ISR RX: lee errores, guarda bytes en rx_buf
6  | }
7  void flush_tx(void) {
8  |    // envía tx_buf por UART
9  | }
10
11 // ===== CLI =====
12 void procesar_comando(char *cmd) {
13 |    // interpreta comandos: stats, led, periodo, retardo
14 | }
15 void tarea_cli(void) {
16 |    // arma línea desde rx_buf y llama procesar_comando
17 | }
18
19 // ===== LED / LOG =====
20 void tarea_led(void) { /* parpadea LED */ }
21 void tarea_log(void) { /* imprime log 1x/seg */ }
22
23 int main() {
24 |    // init LED, pin de traza
25 |    // === setup UART: pines, baudrate, IRQ ===
26 |    uart_init(...);
27 |    irq_set_exclusive_handler(UART0_IRQ, on_uart_rx);
28 |
29 |    while (true) {
30 |        // traza ON
31 |        // led / log / cli / flush_tx
32 |        // traza OFF

```

Código raspberry pi pico w (secundaria)

```

1  // ===== Pico puente: pasarela USB <-> UART =====
2
3  int main() {
4  |    stdio_init_all(); // inicia USB
5  |    sleep_ms(2000); // espera a que la consola USB conecte
6  |
7  |    // === Config UART (pines, 115200 8N1, FIFO) ===
8  |    uart_init(UART_ID, BAUD_RATE);
9  |    gpio_set_function(UART_TX_PIN, GPIO_FUNC_UART);
10 |    gpio_set_function(UART_RX_PIN, GPIO_FUNC_UART);
11 |    uart_set_format(UART_ID, 8, 1, UART_PARITY_NONE);
12 |    uart_set_fifo_enabled(UART_ID, true);
13 |
14 |    while (true) {
15 |        // USB -> UART: si llega byte por USB, lo manda por UART
16 |        int c_usb = getchar_timeout_us(1000);
17 |        if (c_usb != PICO_ERROR_TIMEOUT) {
18 |            uart_putc_raw(UART_ID, (char)c_usb);
19 |        }
20 |
21 |        // UART -> USB: todo lo que llegue por UART se imprime por USB
22 |        while (uart_is_readable(UART_ID)) {
23 |            putchar(uart_getc(UART_ID));
24 |        }
25 |
26 |        fflush(stdout); // envía ya lo acumulado
27 |        tight_loop_contents(); // yield mínimo del loop
28 |    }
29 | }

```

Resultados

Respuestas de los Comandos

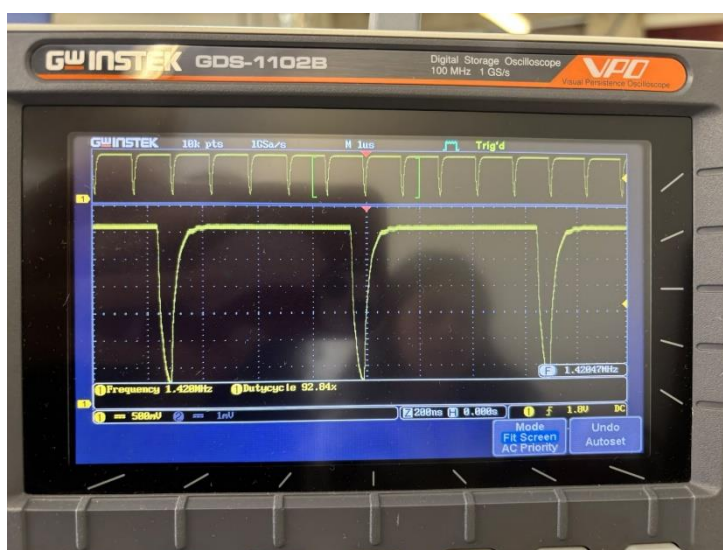
Comando enviado	Respuesta de la Pico	Estado
get stats	overruns: 0, framing errors: 0, descartados: 0	Válido
set led on	ok	Válido
set led off	ok	Válido
set periodo <2000>	error: parametro invalido	Error de sintaxis
set periodo 1000	ok	Válido
set periodo 2000	ok	Válido
desconocido	error: comando desconocido	Comando no reconocido

Tabla de medición

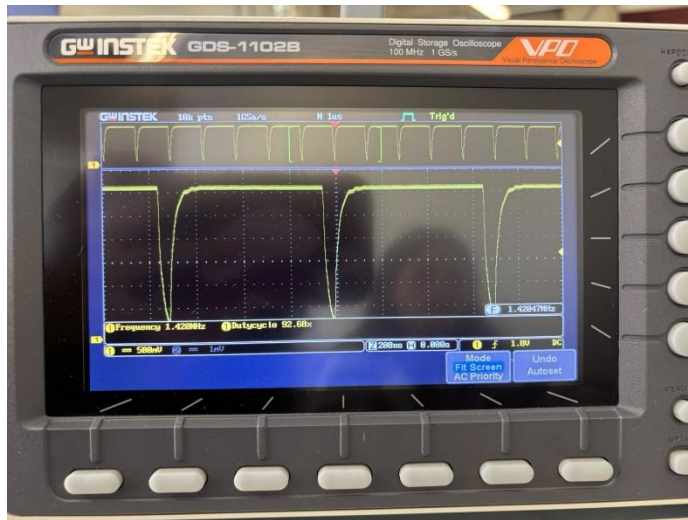
Retardo inyectado	Vuelta medida en traza (teórica / osciloscopio)	Overruns en 30 s	Condición
0 ms	$\approx 32 \mu s$	0	Dentro del límite (< 2.8 ms)
1 ms	$\approx 1.03 \text{ ms}$	0	Dentro del límite (< 2.8 ms)
3 ms	$\approx 3.03 \text{ ms}$	> 0 (si hay tráfico continuo)	Supera el límite de 2.8 ms
10 ms	$\approx 10.03 \text{ ms}$	Pierde datos constantemente	Supera ampliamente la FIFO

Evidencias

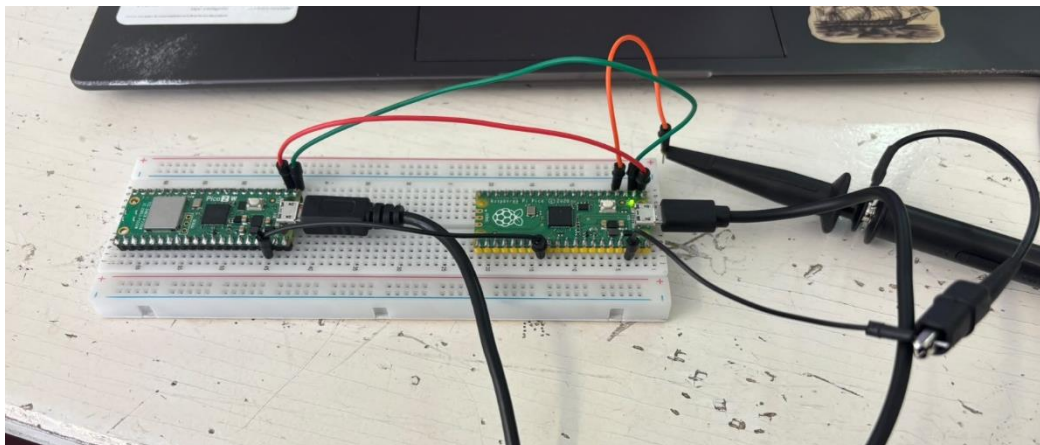
ambas picos en 115200



pico de apoyo en 57000



Armado



Conclusiones

La práctica demuestra que en los sistemas embebidos sin un sistema operativo de tiempo real complejo, el diseño basado en **interrupciones (ISR)**, **buffers circulares** y un **scheduler cooperativo** es fundamental para mantener la concurrencia y evitar la pérdida de datos en periféricos asíncronos como el UART.